

Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Algorytmy i Złożoność Obliczeniowa

Projekt 2 — Algorytmy grafowe

Ocena docelowa: 5.0

Autor: Ivan Taborskykh 288347

Prowadzący: mgr inż. Damian Mroziński

Data: 25.06.2026

Spis treści

1	Wstęp	2
2	Opis zaimplementowanych algorytmów	2
2.1	Algorytm Prima (MST)	2
2.2	Algorytm Kruskala (MST)	2
2.3	Algorytm Dijkstry (SP)	3
2.4	Algorytm Forda–Bellmana (SP)	3
2.5	Algorytm Forda–Fulkersona (MF)	3
3	Opis struktur danych	3
4	Metodologia badań	4
5	Wyniki badań	5
5.1	Badanie A — wpływ rozmiaru grafu	5
5.1.1	Problem MST (Prim, Kruskal)	5
5.1.2	Problem SP (Dijkstra, Forda–Bellmana)	7
5.1.3	Problem MF (Forda–Fulkersona)	8
5.2	Badanie B — wpływ gęstości grafu	9
5.2.1	Problem MST (Prim, Kruskal)	9
5.2.2	Problem SP (Dijkstra, Forda–Bellmana)	10
5.2.3	Problem MF (Forda–Fulkersona)	11
5.3	Porównanie z gotowym rozwiązaniem (duży dataset)	11
6	Wnioski	12

1 Wstęp

Celem projektu była implementacja algorytmów grafowych dla trzech problemów — wyznaczania minimalnego drzewa rozpinającego (MST), najkrótszej ścieżki (SP) oraz maksymalnego przepływu (MF) — a także zbadanie ich wydajności w zależności od rozmiaru i gęstości grafu oraz zastosowanej reprezentacji.

Zaimplementowano pięć algorytmów: dla MST — *algorytm Prima* i *algorytm Kruskala*; dla SP — *algorytm Dijkstry* i *algorytm Forda–Bellmana*; dla MF — *algorytm Forda–Fulkersona*. Każdy algorytm działa na dwóch reprezentacjach grafu: macierzy incydencji oraz liście sąsiedztwa.

Wszystkie algorytmy zaimplementowano w C++ wersji 17 z użyciem szablonów (`template`) oraz `if constexpr`, co umożliwiło jednolitą obsługę obu reprezentacji bez powielania kodu. Wszystkie struktury danych zaimplementowano ręcznie, z dynamiczną alokacją i zwalnianiem pamięci, bez użycia kontenerów STL. Czas działania mierzono wyłącznie dla samego algorytmu (bez wczytywania danych z pliku, losowania grafu czy zapisu wyników) przy użyciu `std::chrono::steady_clock::now()`, a wynik rzutowano na `std::chrono::microseconds`. Każdy wariant powtórzono 67 razy, a jako wynik przyjmowano średnią arytmetyczną, zachowując również minimum i maksimum.

Link do GitHub [link](#)

2 Opis zaimplementowanych algorytmów

2.1 Algorytm Prima (MST)

Algorytm Prima buduje minimalne drzewo rozpinające, zaczynając od jednego wierzchołka i dołączając w każdym kroku najtańszą krawędź łączącą drzewo z wierzchołkiem spoza drzewa. W zaimplementowanej wersji wybór wierzchołka o najmniejszym kluczu odbywa się przez liniowe przeszukanie tablicy kluczy (bez kolejki priorytetowej).

Złożoność wynosi $O(V^2)$ dla listy sąsiedztwa (dominuje V -krotne liniowe wyszukiwanie minimum) [1]. Dla macierzy incydencji koszt jest znacznie wyższy: znalezienie sąsiada danej krawędzi wymaga przeszukania kolumny, więc eksploracja sąsiedztwa kosztuje $O(V \cdot E)$ przy każdym kroku — co czyni tę reprezentację wyraźnie wolniejszą.

2.2 Algorytm Kruskala (MST)

Algorytm Kruskala sortuje wszystkie krawędzie rosnąco według wagi, a następnie dodaje je kolejno do drzewa, pomijając te, które tworzyłyby cykl. Do wykrywania cykli użyto struktury Union–Find (zbiory rozłączne) z kompresją ścieżek i łączeniem według rangi. Do sortowania krawędzi wykorzystano *Shell Sort* z sekwencją Knutha, przeniesiony z projektu 1.

Złożoność zdominowana jest przez sortowanie krawędzi: $O(E^{3/2})$ dla użytej sekwencji Knutha; operacje Union–Find mają koszt niemal liniowy $O(E \alpha(V))$. Wydobycie listy krawędzi z macierzy incydencji kosztuje dodatkowo $O(V \cdot E)$, a z listy sąsiedztwa $O(V + E)$.

2.3 Algorytm Dijkstry (SP)

Algorytm Dijkstry wyznacza najkrótsze ścieżki z jednego źródła do wszystkich pozostałych wierzchołków dla grafu o nieujemnych wagach. W każdym kroku wybierany jest nieodwiedzony wierzchołek o najmniejszej odległości (liniowe wyszukiwanie minimum), po czym relaksowane są jego krawędzie wychodzące.

Złożoność wynosi $O(V^2)$ dla listy sąsiedztwa [1]. Dla macierzy incydencji, podobnie jak w algorytmie Prima, odtworzenie sąsiada krawędzi wymaga dodatkowego przeszukania kolumny, co podnosi koszt do rzędu $O(V^2 \cdot E)$.

2.4 Algorytm Forda–Bellmana (SP)

Algorytm Forda–Bellmana relaksuje wszystkie krawędzie grafu $V - 1$ razy, dzięki czemu radzi sobie również z wagami ujemnymi. Po zakończeniu wykonywany jest dodatkowy przebieg: jeśli jakkolwiek krawędź można jeszcze zrelaksować, oznacza to obecność **ujemnego cyklu** — algorytm sygnalizuje wtedy ostrzeżenie.

Złożoność wynosi $O(V \cdot E)$ dla listy sąsiedztwa [1]. Dla macierzy incydencji każdy z $V - 1$ przebiegów wymaga odtworzenia końców krawędzi z kolumn, co podnosi koszt do rzędu $O(V^2 \cdot E)$ — stąd ta reprezentacja jest dla tego algorytmu skrajnie wolna.

2.5 Algorytm Forda–Fulkersona (MF)

Algorytm Forda–Fulkersona wyznacza maksymalny przepływ między źródłem a ujściem, wielokrotnie szukając ścieżki powiększającej w sieci residualnej i przepychając wzdłuż niej przepływ równy minimalnej przepustowości na ścieżce. Do szukania ścieżek użyto przeszukiwania wszerz (BFS), co odpowiada wariantowi Edmondsa–Karpa. Sieć residualna przechowywana jest jako macierz $V \times V$.

Klasyczny wariant Edmondsa–Karpa (z przeszukiwaniem listowym) ma złożoność $O(V \cdot E^2)$ [1]. W tej implementacji BFS operuje jednak na macierzy residualnej $V \times V$, więc pojedyncze przeszukanie kosztuje $O(V^2)$, a liczba ścieżek powiększających jest rzędu $O(V \cdot E)$ — co daje łączny koszt rzędu $O(V^3 \cdot E)$. Budowa sieci residualnej kosztuje $O(V \cdot E)$ dla macierzy incydencji oraz $O(V + E)$ dla listy sąsiedztwa.

3 Opis struktur danych

Obie reprezentacje grafu zaimplementowano ręcznie, z dynamiczną alokacją i zwalnianiem pamięci, bez użycia kontenerów STL. Po wczytaniu grafu program weryfikuje poprawność: liczba krawędzi w zbudowanej strukturze jest porównywana z liczbą podaną w pliku wejściowym.

Macierz incydencji Tablica dwuwymiarowa o wymiarach $V \times E$, gdzie wiersze odpowiadają wierzchołkom, a kolumny krawędziom. Dla grafu skierowanego kierunek krawędzi kodowany jest znakiem wagi (wierzchołek początkowy ma wartość $+w$, końcowy $-w$); dla nieskierowanego oba końce mają wartość $+w$. Sprawdzenie sąsiedztwa wymaga przeszukania kolumny.

Lista sąsiedztwa Tablica V wskaźników na jednokierunkowe listy następników; każdy węzeł przechowuje numer sąsiada oraz wagę krawędzi. Dla grafu nieskierowanego

każda krawędź zapisywana jest dwukrotnie. Sąsiedzi danego wierzchołka są dostępni bezpośrednio.

4 Metodologia badań

Środowisko testowe było zgodne ze specyfikacją z treści projektu: system Xubuntu 25.10 wirtualizowany za pomocą VirtualBox 7.2.4, kompilator gcc 15.2.0 oraz system budowania cmake 3.31.6. Program kompilowano z flagami `-Wall -Wextra -Werror` (bez ostrzeżeń).

Do obsługi parametrów wejściowych użyto biblioteki dostarczonej przez prowadzącego (`libparameters.so`). Plik `CMakeLists.txt` wyróżnia lokalizacje kodu, nagłówków i biblioteki, a dołączanie odpowiednich ścieżek pozostawiono cmake'owi.

Grafy generowano jako spójne: najpierw losowane jest drzewo rozpinające (każdy nowy wierzchołek łączony z losowym już dodanym), a następnie dokładane są losowe krawędzie aż do uzyskania wymaganej gęstości. Zakres wag zależy od liczby krawędzi (`rand(1, ⌊ $E \cdot 4/5$ ⌋)`), co dopuszcza powtarzające się wagi.

Dla każdego wariantu wykonywano 67 niezależnych pomiarów; graf generowano od nowa przed każdym pomiarem. Mierzono wyłącznie czas działania algorytmu (bez wczytywania, losowania i zapisu). Wyniki agregowano jako: średnia, minimum i maksimum czasu wykonania.

W **badaniu A** (wpływ rozmiaru grafu) testowano pięć rozmiarów $n \in \{10, 20, 50, 100, 200\}$ przy stałej gęstości 50%.

W **badaniu B** (wpływ gęstości) testowano gęstości $\{25, 50, 75\}\%$ przy stałym rozmiarze $n = 100$. Dla problemów SP i MF grafy są skierowane, dla MST — nieskierowane.

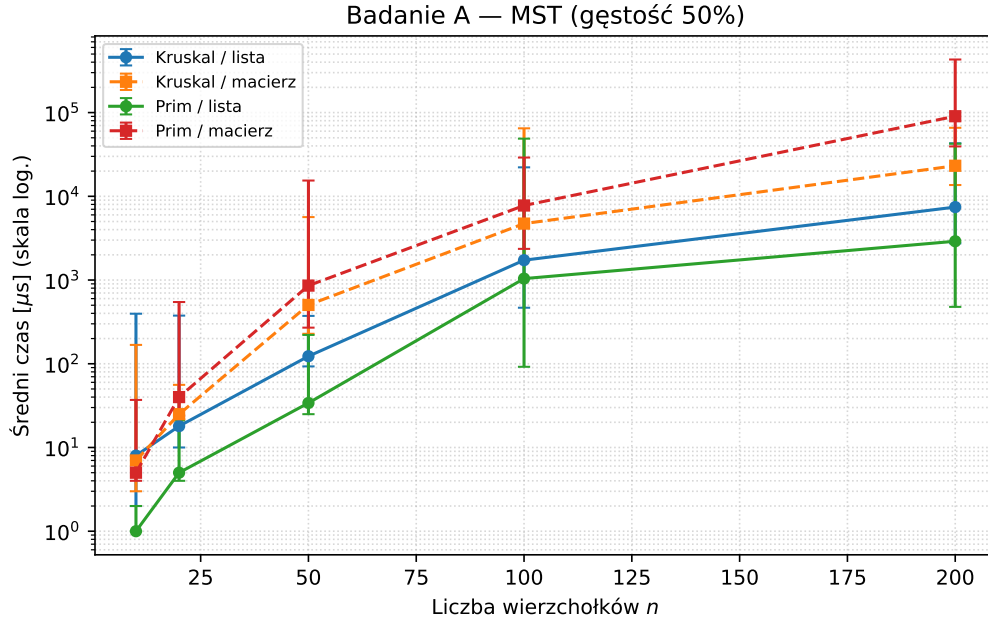
5 Wyniki badań

5.1 Badanie A — wpływ rozmiaru grafu

5.1.1 Problem MST (Prim, Kruskal)

Tabela 1: Czas [μs] (średnia, min, max) — MST, badanie A (gęstość 50%).

Algorytm	Struktura	n	Średnia [μs]	Min [μs]	Max [μs]
Prim	lista	10	1	1	2
		20	5	4	21
		50	34	25	221
		100	1 040	92	48 823
		200	2 905	478	42 455
Prim	macierz	10	5	4	37
		20	40	24	546
		50	858	270	15 429
		100	7 764	2 356	29 039
		200	90 459	39 424	430 529
Kruskal	lista	10	8	2	396
		20	18	10	376
		50	123	93	373
		100	1 726	467	22 180
		200	7 435	2 745	43 003
Kruskal	macierz	10	7	3	168
		20	25	22	56
		50	504	228	5 646
		100	4 722	1 686	64 854
		200	23 165	13 641	65 822



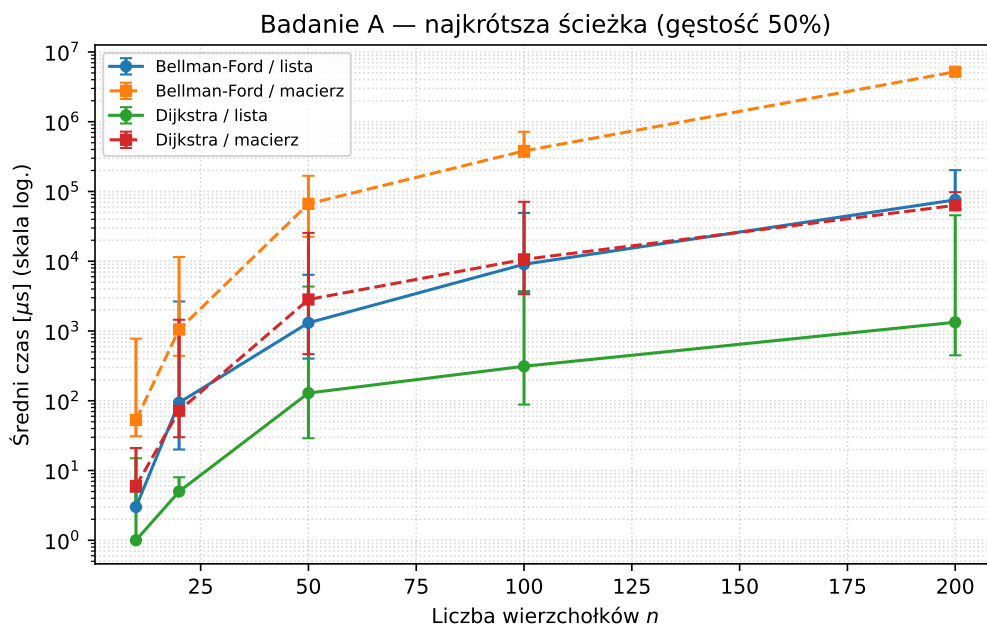
Rysunek 1: MST — średni czas w zależności od liczby wierzchołków (oś Y logarytmiczna, słupki błęd: min/max).

Lista sąsiedztwa jest wyraźnie szybsza od macierzy incydencji dla obu algorytmów, a różnica rośnie z rozmiarem grafu. Przy $n = 200$ Prim na liście ($\approx 2905 \mu s$) jest około 31-krotnie szybszy niż na macierzy ($\approx 90459 \mu s$) — na macierzy każda relaksacja wymaga przeszukania kolumn w poszukiwaniu sąsiada, co daje koszt rzędu $O(V^2 \cdot E)$. Na liście Prim wyprzedza Kruskala (2905 wobec $7435 \mu s$), ponieważ Kruskal musi posortować wszystkie krawędzie. Na macierzy zależność się odwraca — Kruskal ($23165 \mu s$) jest szybszy od Prima, bo wydobywa krawędzie jednym przejściem, podczas gdy Prim wielokrotnie skanuje macierz.

5.1.2 Problem SP (Dijkstra, Forda–Bellmana)

Tabela 2: Czas [μs] (średnia, min, max) — najkrótsza ścieżka, badanie A (gęstość 50%).

Algorytm	Struktura	n	Średnia [μs]	Min [μs]	Max [μs]
Dijkstra	lista	10	1	1	15
		20	5	5	8
		50	129	29	4 334
		100	312	88	3 728
		200	1 335	448	45 622
Dijkstra	macierz	10	6	5	21
		20	72	30	1 449
		50	2 828	466	25 496
		100	10 630	3 371	71 112
		200	63 288	54 762	97 713
Forda–Bellmana	lista	10	3	3	5
		20	94	20	2 650
		50	1 310	402	6 395
		100	9 046	3 595	49 301
		200	75 777	56 446	202 215
Forda–Bellmana	macierz	10	53	31	775
		20	1 050	441	11 501
		50	66 498	22 465	167 564
		100	381 381	313 843	717 673
		200	5 199 232	5 020 038	5 781 255



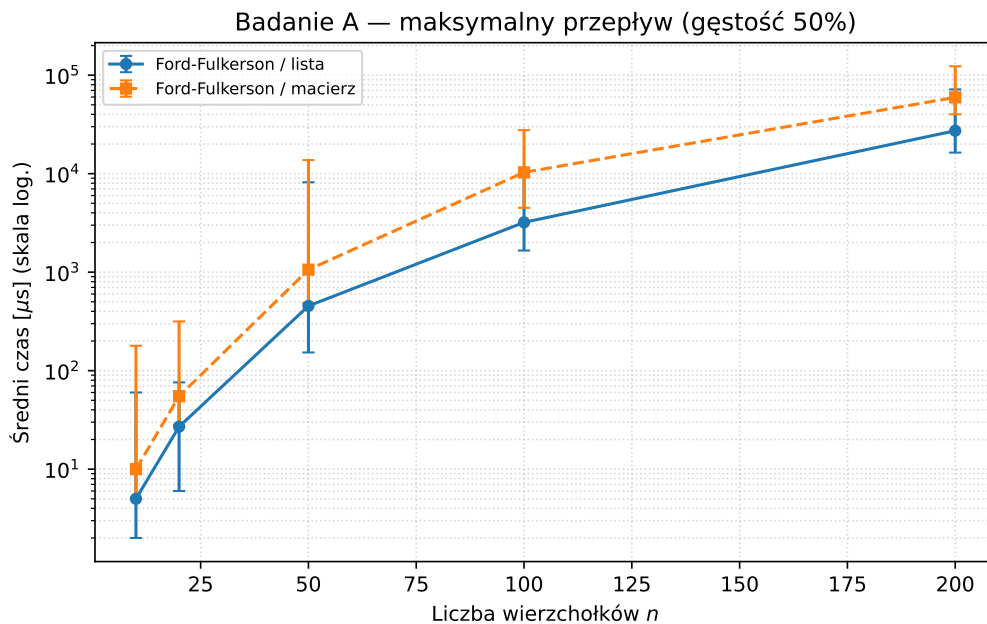
Rysunek 2: Najkrótsza ścieżka — średni czas w zależności od liczby wierzchołków (oś Y logarytmiczna, słupki błędów: min/max).

Dijkstra na liście jest najszybszy ($\approx 1\,335\ \mu\text{s}$ przy $n = 200$). Forda–Bellmana na macierzy incydencji wypada katastrofalnie — $\approx 5,2$ sekundy przy $n = 200$, czyli blisko 3900-krotnie wolniej niż Dijkstra na liście. Wynika to z kosztu $O(V^2 \cdot E)$: w każdym z $V - 1$ przebiegów dla każdej krawędzi trzeba odtworzyć jej końce, skanując kolumnę macierzy. Lista sąsiedztwa redukuje ten koszt do $O(V \cdot E)$, dlatego Forda–Bellmana na liście ($75\,777\ \mu\text{s}$) jest o rzędy wielkości szybszy niż na macierzy.

5.1.3 Problem MF (Forda–Fulkersona)

Tabela 3: Czas [μs] (średnia, min, max) — maksymalny przepływ, badanie A (gęstość 50%).

Algorytm	Struktura	n	Średnia [μs]	Min [μs]	Max [μs]
Forda–Fulkersona	lista	10	5	2	60
		20	27	6	76
		50	454	153	8 174
		100	3 203	1 658	9 281
		200	27 292	16 368	71 702
Forda–Fulkersona	macierz	10	10	5	179
		20	55	28	316
		50	1 060	485	13 750
		100	10 310	4 505	27 701
		200	59 530	40 078	123 176



Rysunek 3: Maksymalny przepływ — średni czas w zależności od liczby wierzchołków (oś Y logarytmiczna, słupki błęd: min/max).

Również tutaj lista sąsiedztwa jest szybsza (około 2,2-krotnie przy $n = 200$: 27 292 wobec 59 530 μs). Różnica jest mniejsza niż przy MST i SP, ponieważ rdzeń algorytmu (BFS

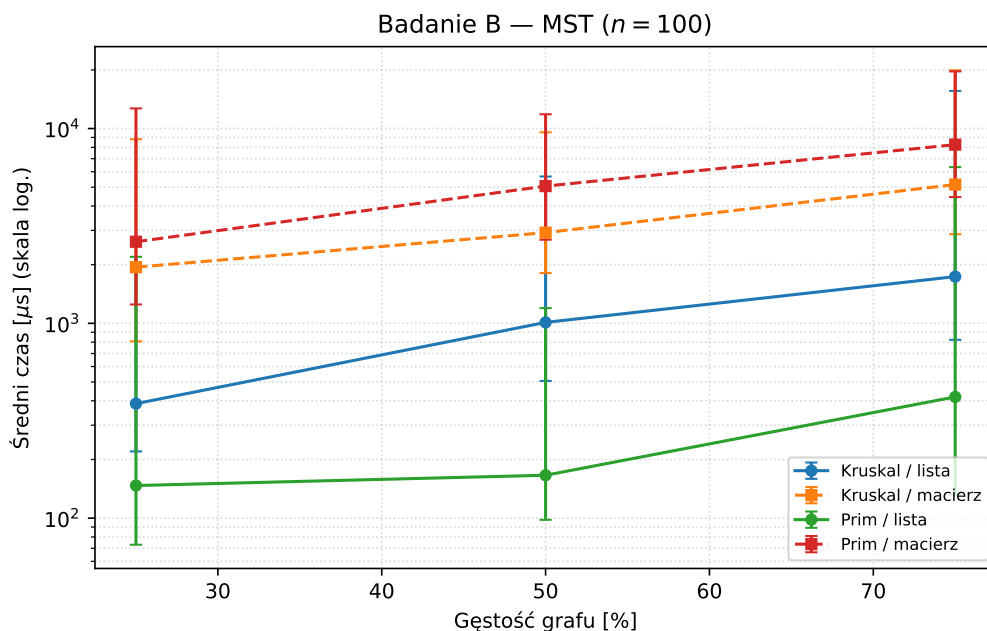
po sieci residualnej $V \times V$) jest wspólny dla obu reprezentacji — różni je jedynie koszt jednorazowej budowy sieci residualnej.

5.2 Badanie B — wpływ gęstości grafu

5.2.1 Problem MST (Prim, Kruskal)

Tabela 4: Czas [μ s] (średnia, min, max) — MST, badanie B ($n = 100$).

Algorytm	Struktura	Gęstość	Średnia [μ s]	Min [μ s]	Max [μ s]
Prim	lista	25%	147	73	2 195
		50%	166	98	1 197
		75%	419	132	6 338
Prim	macierz	25%	2 623	1 250	12 686
		50%	5 063	2 687	11 838
		75%	8 268	4 450	19 647
Kruskal	lista	25%	387	220	2 024
		50%	1 011	506	5 672
		75%	1 737	823	15 601
Kruskal	macierz	25%	1 942	808	8 814
		50%	2 921	1 811	9 563
		75%	5 158	2 869	19 891



Rysunek 4: MST — średni czas w zależności od gęstości grafu ($n = 100$; oś Y logarytmiczna, słupki błędów: min/max).

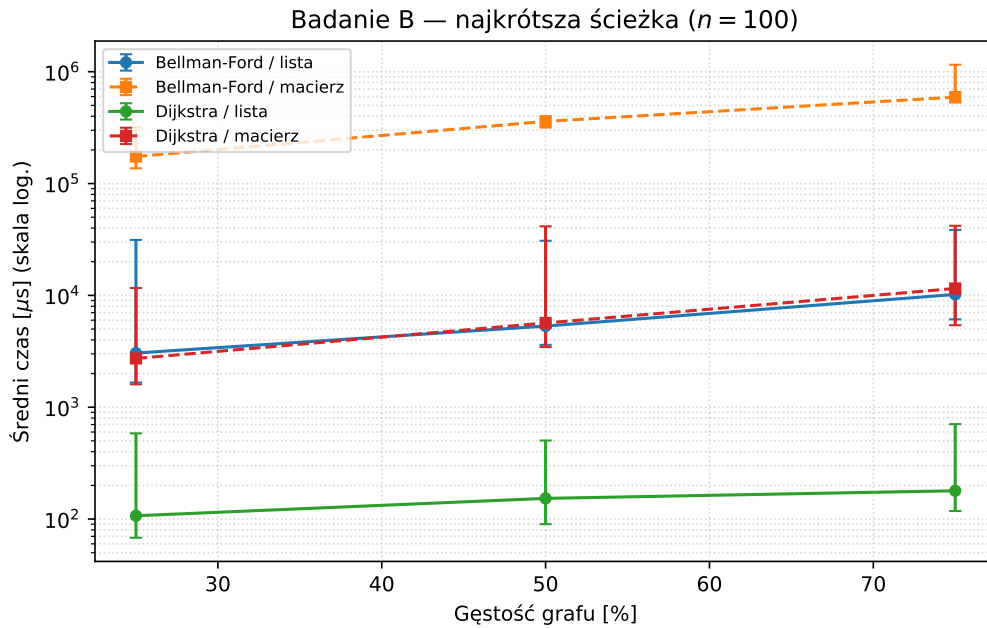
Wraz ze wzrostem gęstości (a więc liczby krawędzi) czas rośnie dla obu algorytmów i obu reprezentacji. Na liście wartości są znacznie mniejsze niż na macierzy, lecz również

rosną z gęstością. Czas na macierzy zależy głównie od liczby kolumn (E), dlatego rośnie z gęstością najwyraźniej.

5.2.2 Problem SP (Dijkstra, Forda–Bellmana)

Tabela 5: Czas [μ s] (średnia, min, max) — najkrótsza ścieżka, badanie B ($n = 100$).

Algorytm	Struktura	Gęstość	Średnia [μ s]	Min [μ s]	Max [μ s]
Dijkstra	lista	25%	107	68	584
		50%	153	90	504
		75%	179	118	706
Dijkstra	macierz	25%	2 729	1 602	11 645
		50%	5 665	3 455	41 456
		75%	11 525	5 411	41 935
Forda–Bellmana	lista	25%	3 051	1 663	31 335
		50%	5 313	3 614	30 796
		75%	10 196	6 107	38 505
Forda–Bellmana	macierz	25%	174 524	136 846	318 336
		50%	359 485	316 742	398 407
		75%	593 091	532 574	1 155 542



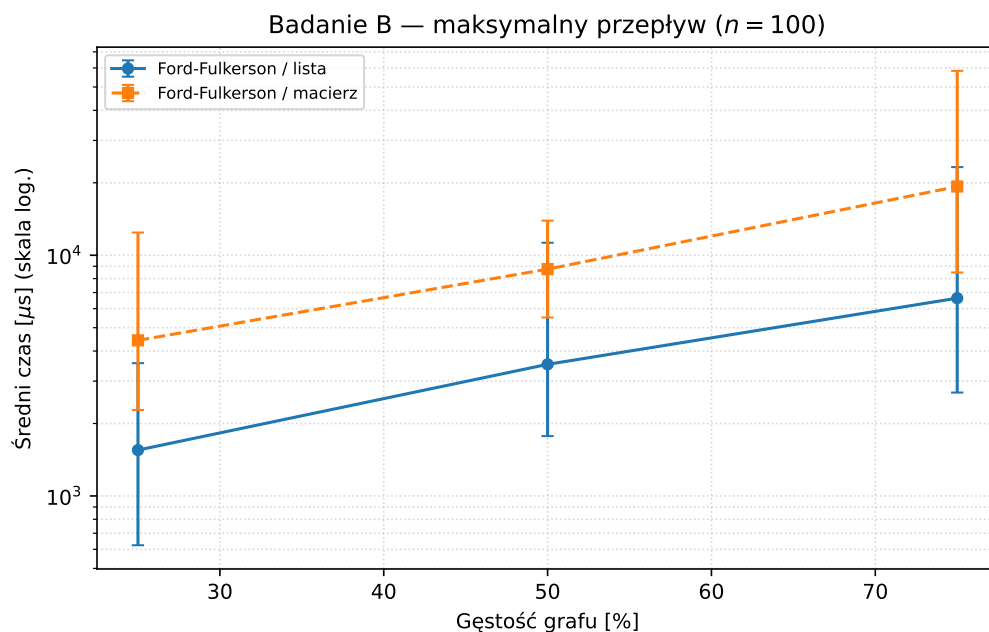
Rysunek 5: Najkrótsza ścieżka — średni czas w zależności od gęstości grafu ($n = 100$; oś Y logarytmiczna, słupki błędów: min/max).

Dijkstra na liście prawie nie reaguje na gęstość (107–179 μ s), ponieważ przy $V = 100$ koszt zdominowany jest przez $O(V^2)$ wyszukiwanie minimum, a nie przez liczbę krawędzi. Forda–Bellmana, którego koszt zależy wprost od E , rośnie z gęstością wyraźnie — na macierzy od ≈ 175 ms do ≈ 593 ms.

5.2.3 Problem MF (Ford–Fulkersona)

Tabela 6: Czas [μs] (średnia, min, max) — maksymalny przepływ, badanie B ($n = 100$).

Algorytm	Struktura	Gęstość	Średnia [μs]	Min [μs]	Max [μs]
Ford–Fulkersona	lista	25%	1 551	624	3 562
		50%	3 522	1 773	11 270
		75%	6 634	2 689	23 243
Ford–Fulkersona	macierz	25%	4 431	2 275	12 427
		50%	8 756	5 516	13 929
		75%	19 306	8 478	58 359



Rysunek 6: Maksymalny przepływ — średni czas w zależności od gęstości grafu ($n = 100$; oś Y logarytmiczna, słupki błędów: min/max).

Czas rośnie z gęstością dla obu reprezentacji, co jest zgodne z zależnością algorytmu od liczby krawędzi (więcej ścieżek powiększających i większa sieć residualna). Lista pozostaje około 2,5–3-krotnie szybsza od macierzy w całym zakresie.

5.3 Porównanie z gotowym rozwiązaniem (duży dataset)

Do badań na dużym grafie użyto zbioru `soc-sign-bitcoinalpha` z repozytorium SNAP (Stanford) — sieci ocen zaufania użytkowników platformy wymiany bitcoina, liczącej 3783 wierzchołki i 24186 krawędzi. Oryginalne oceny (z zakresu $-10 \dots 10$) przekształcono do wag dodatnich ($w = \text{ocena} + 11$, czyli zakres $1 \dots 21$), aby spełnić założenie o naturalnych wagach. Graf jest skierowany. Zbiór pobrano z kolekcji SNAP: snap.stanford.edu/data/index.html.

Na tym grafie uruchomiono obie własne implementacje najkrótszej ścieżki (z wierzchołka 0, na liście sąsiedztwa) i porównano je z biblioteką `networkx` — funkcjami `single_source_dijkstra_path_length` oraz `single_source_bellman_ford_path_length`. Reprezentację macierzową pominięto — macierz 3783×24186 byłaby zbyt duża.

Poprawność. Obie nasze implementacje dały wyniki identyczne z `networkx` dla wszystkich 3747 osiągalnych wierzchołków, a także identyczne ze sobą nawzajem (tabela 7).

Tabela 7: Zgodność wyników własnych implementacji z `networkx` (bitcoin-alpha, źródło: wierzchołek 0).

Implementacja	Porównanych wierzchołków	Zgodnych	Różnic
Dijkstra (własna)	3747	3747	0
Forda–Bellmana (własna)	3747	3747	0

Czas działania. Każdy algorytm zmierzono 50 razy. Własne implementacje zmierzono w tym samym środowisku co badania A i B (Xubuntu w VirtualBox), natomiast `networkx` uruchomiono na systemie gospodarza (Windows z Pythonem), ponieważ w maszynie wirtualnej nie udało się zainstalować tej biblioteki. Z tego powodu czasy bezwzględne należy traktować orientacyjnie, natomiast wnioski jakościowe (które rozwiązanie jest szybsze i dlaczego) pozostają miarodajne. Wyniki zestawiono w tabeli 8.

Tabela 8: Czas wyznaczania najkrótszych ścieżek na zbiorze bitcoin-alpha (50 pomiarów, źródło: wierzchołek 0).

Algorytm	Implementacja	Średnia [μ s]	Min [μ s]	Max [μ s]
Dijkstra	nasza (lista)	77 310	59 601	190 543
	<code>networkx</code>	14 563	11 079	26 640
Forda–Bellmana	nasza (lista)	1 226 449	1 008 947	2 435 489
	<code>networkx</code>	26 677	17 292	43 076

`networkx` okazał się szybszy od naszych implementacji: dla Dijkstry około 5-krotnie (≈ 15 ms wobec ≈ 77 ms), a dla Forda–Bellmana aż około 46-krotnie (≈ 27 ms wobec $\approx 1,23$ s). Wynika to z lepszych struktur danych i wariantów algorytmów: `networkx` używa Dijkstry z kolejką priorytetową (kopcem), o złożoności $O((V + E) \log V)$, podczas gdy nasza implementacja korzysta z liniowego wyszukiwania minimum $O(V^2)$. Jeszcze większa różnica dla Forda–Bellmana wynika z tego, że `networkx` stosuje wariant kolejkowy (przetwarzający tylko wierzchołki, których odległość się zmieniła), natomiast nasza wykonuje pełne $V - 1$ przebiegów po wszystkich krawędziach. Mimo to wyniki obu narzędzi są identyczne — nasza implementacja jest poprawna, lecz prostsza i wolniejsza od zoptymalizowanej biblioteki.

6 Wnioski

Przeprowadzone badania pozwoliły sformułować kilka wniosków dotyczących zaimplementowanych algorytmów grafowych.

Reprezentacja grafu. Lista sąsiedztwa okazała się szybsza od macierzy incydencji dla wszystkich algorytmów i problemów, a przewaga rosła z rozmiarem grafu. Przyczyną jest sposób dostępu do sąsiedztwa: lista udostępnia sąsiadów wierzchołka bezpośrednio, podczas gdy macierz incydencji wymaga przeszukiwania kolumn w poszukiwaniu drugiego końca krawędzi. Najdobitniej widać to dla algorytmu Forda–Bellmana, gdzie macierz osiągnęła koszt rzędu $O(V^2 \cdot E)$ i czas $\approx 5,2$ s przy $n = 200$, wobec ≈ 76 ms na liście.

Wybór algorytmu MST. Na liście sąsiedztwa Prim był szybszy od Kruskala (brak kosztownego sortowania krawędzi). Na macierzy incydencji zależność się odwracała — Kruskal wygrywał, ponieważ wydobywa krawędzie jednym przejściem, a Prim wielokrotnie skanuje macierz.

Wybór algorytmu SP. Dijkstra był zdecydowanie szybszy od Forda–Bellmana w każdym przypadku. Forda–Bellmana ma jednak przewagę funkcjonalną — obsługuje wagi ujemne i wykrywa ujemne cykle, czego Dijkstra nie potrafi.

Wpływ gęstości. Czas algorytmów zależnych od liczby krawędzi (Ford–Bellmana, Ford–Fulkersona, algorytmy na macierzy) rósł wraz z gęstością. Dijkstra na liście był na gęstość niemal niewrażliwy, ponieważ przy małym V dominuje składnik $O(V^2)$.

Poprawność. Wszystkie implementacje przeszły weryfikację: algorytmy MST i SP dawały zgodne wyniki między reprezentacjami, a na dużym zbiorze rzeczywistym (bitcoin-alpha) własne implementacje Dijkstry i Forda–Bellmana były w 100% zgodne z biblioteką networkx. Biblioteka okazała się jednak istotnie szybsza (Dijkstra ≈ 5 -krotnie, Ford–Bellmana ≈ 46 -krotnie) dzięki lepszym strukturom danych (kopiec, wariant kolejkowy), co dodatkowo podkreśla znaczenie ich doboru.

Literatura

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022. Rozdziały 21–24 (MST, najkrótsze ścieżki) oraz 26 (przepływy).